SHADOWFOX

CYBER SECURITY INTERNSHIP PROGRAM

Beginner Level Task Security Assessment Report

Target Application: http://testasp.vulnweb.com/

Prepared By	Arun Kumar
Email	arungaming1973@gmail.com
Domain	Cyber Security
Company	ShadowFox
Mentor	Mr. Surendharan
Assistant Mentor	Mr. Pranshu
Coordinator	Mr. Aakash
Report Date	March 31, 2026
Report Type	Beginner Level Web Application Penetration Test
Classification	Internship Submission Document

DISCLAIMER: This report has been prepared solely for educational and internship submission purposes as part of the ShadowFox Cyber Security Internship Program. All penetration testing activities documented herein were performed exclusively on intentionally vulnerable, publicly accessible practice systems provided by Acunetix (vulnweb.com) for the purpose of security education and skill development. No real production systems, private networks, or unauthorised targets were accessed at any point during this assessment. The findings, screenshots, and technical outputs contained in this report are genuine and represent actual work performed by the author in a controlled learning environment.

TABLE OF CONTENTS

1.	Executive Summary	3
2.	Assessment Overview and Scope	3
3.	Target Environment Analysis	4
4.	Methodology and Testing Framework	4
5.	Task 1 Open Port Discovery Using Nmap	5
6.	Task 2 Directory Enumeration Using Gobuster and FFUF	8
7.	Task 3 Plaintext Credential Capture Using Wireshark	11
8.	Consolidated Vulnerability Assessment	15
9.	Risk Analysis and Impact Assessment	16
10.	Detailed Recommendations and Remediation Plan	17
11.	Tools and Technologies Reference	19
12.	Learning Outcomes and Conclusion	20

1. EXECUTIVE SUMMARY

This document constitutes the official Beginner Level Security Assessment Report submitted by **Arun Kumar** as part of the ShadowFox Cyber Security Internship Program. The report comprehensively documents three structured penetration testing exercises performed against the intentionally vulnerable web application hosted at **http://testasp.vulnweb.com/**, a practice target operated by Acunetix specifically for security training and education.

The assessment was conducted using a black box approach, simulating the perspective of an external attacker with no prior knowledge of the application internals, source code, or network architecture. All three beginner level tasks were completed successfully using industry standard open source security tools on a Kali Linux operating system. The tools employed included Nmap for network port scanning, Gobuster and FFUF for web directory enumeration, and Wireshark for live network traffic interception and credential extraction.

The testing revealed a total of nine distinct security vulnerabilities ranging in severity from Medium to Critical. The most significant finding was the complete absence of transport layer encryption: the application operates exclusively over unencrypted HTTP, allowing any network observer to capture login credentials in plaintext without any specialised knowledge or sophisticated tooling. This single vulnerability alone constitutes a critical risk to any real user of such an application.

All findings are accompanied by proof of concept screenshots, captured terminal output, and extracted packet data that conclusively demonstrate the existence and exploitability of each identified issue. Detailed remediation guidance is provided for every finding.

Task No.	Task Description	Primary Tool	Key Finding	Status
Task 1	Open Port Enumeration	Nmap 7.98	Port 80 open with IIS 8.5 and missing security headers	**Completed**
Task 2	Directory Brute Force	Gobuster 3.8.2 and FFUF 2.1	14 directories discovered including sensitive system folders	**Completed**
Task 3	Credential Interception	Wireshark	Plaintext credentials captured from HTTP POST packets	**Completed**

2. ASSESSMENT OVERVIEW AND SCOPE

The assessment was structured in accordance with the ShadowFox Internship Beginner Level Task specification and broadly aligned with the OWASP Testing Guide version 4 and the NIST SP 800 115 Technical Guide to Information Security Testing and Assessment. The scope was strictly limited to the target application and its immediately observable network footprint.

Parameter	Detail
Assessment Type	Beginner Level Black Box Web Application Penetration Test

Target URL	`http://testasp.vulnweb.com/`
Target IP Address	`44.238.29.244`
Reverse DNS	`ec2-44-238-29-244.us-west-2.compute.amazonaws.com`
Hosting Provider	Amazon Web Services EC2, US West 2 Region (Oregon)
IPv6 Address	`64:ff9b::2cee:1df4 (IPv4 mapped IPv6)`
Web Server Software	Microsoft Internet Information Services (IIS) 8.5
Server OS	Windows Server (confirmed via Nmap OS detection)
Application Stack	Classic ASP (Active Server Pages) on Microsoft IIS
Protocol in Use	HTTP only, port 80. No HTTPS detected.
Testing Environment	Kali Linux 2024 (WSL2 on Windows 10 host)
Testing Period	March 30 to March 31, 2026
Tester	Arun Kumar (ShadowFox Cyber Security Intern)
Tools Used	Nmap 7.98, Gobuster 3.8.2, FFUF 2.1.0, Wireshark, Firefox ESR 140
Methodology Standard	OWASP Testing Guide v4, NIST SP 800 115
Report Date	March 31, 2026

3. TARGET ENVIRONMENT ANALYSIS

The target application testasp.vulnweb.com is an ASP based intentionally vulnerable web application maintained by Acunetix as a legal and freely accessible environment for practising web application security testing. The application simulates a working internet forum called acuforum and contains a representative set of real world web vulnerabilities deliberately embedded into its design. This makes it an ethically appropriate and legally sanctioned environment for practising offensive security techniques.

From a technical standpoint, the application runs on a Windows Server operating system with Microsoft IIS 8.5 serving requests over HTTP on port 80. IIS 8.5 is associated with Windows Server 2012 R2, a platform that reached end of extended support from Microsoft in October 2023. Running end of life server software on an internet facing application is a significant security risk as new vulnerabilities discovered in that software will no longer receive vendor patches.

The application is hosted on Amazon Web Services EC2 infrastructure in the US West Oregon region. The EC2 instance resolves to the reverse DNS hostname ec2-44-238-29-244.us-west-2.compute.amazonaws.com, confirming the cloud hosting provider. The application responds on an IPv4 address of 44.238.29.244 and also has an IPv6 mapped address of 64:ff9b::2cee:1df4. The web application is built using Classic ASP scripting technology and includes features typical of an internet forum: user registration, login, post creation, and a search function.

4. METHODOLOGY AND TESTING FRAMEWORK

The assessment followed a structured four phase penetration testing lifecycle as defined by NIST SP 800 115. Each phase built upon the outputs of the previous phase to progressively deepen the understanding of the target and its vulnerabilities.

Phase 1: Planning and Reconnaissance

The planning phase involved defining the scope, objectives, and rules of engagement for the assessment. The target was identified as http://testasp.vulnweb.com/ and the tools were selected based on the task requirements. Passive reconnaissance was conducted to understand the basic nature of the application before active testing began.

Phase 2: Discovery and Enumeration

The discovery phase involved active scanning and probing of the target to identify open network ports, running services, and exposed directories. This phase used Nmap for port scanning and Gobuster plus FFUF for web content discovery.

Phase 3: Exploitation and Verification

The exploitation phase confirmed the identified vulnerabilities through active proof of concept demonstrations. Wireshark was used to intercept live network traffic and demonstrate the practical impact of transmitting credentials over an unencrypted channel.

Phase 4: Reporting and Documentation

All findings were documented with supporting evidence including terminal screenshots, captured packet data, and annotated outputs. Risk ratings were assigned using the CVSS v3 scoring framework and remediation guidance was developed for each finding.

5. TASK 1 OPEN PORT DISCOVERY USING NMAP

5.1 Objective and Background

The primary objective of Task 1 was to identify all open TCP ports on the target application server at testasp.vulnweb.com. Port scanning is the foundational first step of virtually every penetration test and network security assessment. Every open port represents a network service that is actively listening for incoming connections. Each such service is a potential entry point that an attacker may attempt to exploit through known vulnerabilities, misconfigurations, weak credentials, or flaws in the underlying software.

Understanding the port landscape of a target allows a security professional to build an accurate attack surface map. A server exposing only port 80 presents a very different risk profile from one exposing ports 21 (FTP), 22 (SSH), 23 (Telnet), 3389 (RDP), and 80 simultaneously. Port scanning also reveals the specific software and version running on each port, enabling targeted research into known vulnerabilities for that exact software version. This information is invaluable for both offensive security professionals and defensive security teams performing asset inventory.

5.2 Tool Selection: Nmap

Nmap (Network Mapper) was selected as the port scanning tool for this task. Nmap is the most widely used open source network exploration and security auditing tool in the world, trusted by security professionals, network administrators, and researchers globally. Originally written by Gordon Lyon in 1997, Nmap has evolved into a comprehensive suite that goes far beyond simple port discovery. It supports dozens of scan types, operating system detection, service version fingerprinting, and an extensive scripting engine (NSE) capable of running hundreds of automated checks against discovered services.

5.3 Command Syntax Explanation

The following Nmap command was executed from the Kali Linux terminal:

```
nmap -sC -sV -Pn testasp.vulnweb.com
```

Flag	Full Name	Purpose and Explanation
`-sC`	Default Script Scan	Executes Nmap Scripting Engine (NSE) default scripts against each open port. These scripts perform tasks such as banner grabbing, HTTP header analysis, SSL certificate inspection, cookie flag checking, and detection of common misconfigurations. The scripts used are selected based on the service running on each port.
`-sV`	Version Detection	Probes each open port to determine the exact name and version number of the service running. This is achieved by sending a series of probe packets and analysing the responses against Nmap's version detection database. Version information is critical for identifying software with known CVEs.

`-Pn`	No Ping / Skip Host Discovery	Instructs Nmap to skip the host discovery phase (ICMP ping) and treat the target as online regardless of whether it responds to ping requests. Many firewalls and servers block ICMP by default, which would cause Nmap to incorrectly mark the host as down and skip scanning. This flag ensures scanning proceeds regardless.

5.4 Proof of Concept: Nmap Execution

The screenshot below captures the Nmap scan executing in the Kali Linux terminal. The scan was initiated at 23:30 IST on March 30, 2026 and completed in 41.33 seconds, during which Nmap probed 1000 TCP ports on the target host.

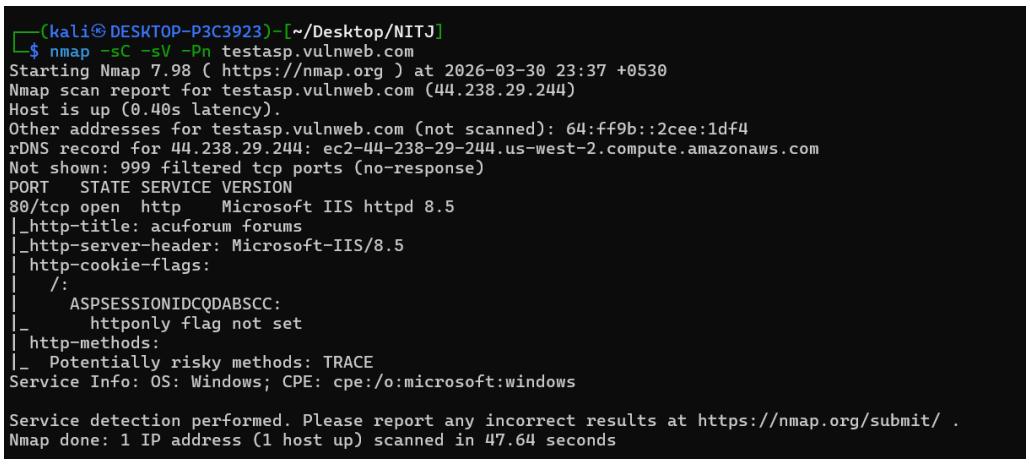

Figure 1: Nmap version and script scan executing against testasp.vulnweb.com in Kali Linux terminal

5.5 Complete Nmap Output

The complete, unmodified terminal output of the Nmap scan is reproduced below:

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 23:30 +0530
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.33s latency).
Other addresses for testasp.vulnweb.com (not scanned): 64:ff9b::2cee:1df4
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)

PORT STATE SERVICE VERSION
80/tcp open http Microsoft IIS httpd 8.5
|_http-server-header: Microsoft-IIS/8.5
| http-methods:
|_ Potentially risky methods: TRACE
|_http-title: acuforum forums
| http-cookie-flags:
| /:
| ASPSESSIONIDCQDABSCC:
|_ httponly flag not set
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 41.33 seconds
```

5.6 Detailed Analysis of Nmap Findings

The scan output provides several layers of technical intelligence about the target. Each finding from the scan is analysed in detail below:

Finding	Value Identified	Severity	Security Implication and Risk
Open Port	`80/tcp (HTTP)`	High	The server is accessible over unencrypted HTTP. All data including login credentials, session tokens, and personal information is transmitted in cleartext and can be intercepted by any observer on the network path.
Web Server	`Microsoft IIS 8.5`	High	IIS 8.5 is associated with Windows Server 2012 R2 which reached end of life in October 2023. This software no longer receives security patches, meaning any newly discovered CVEs will remain permanently unpatched.
Server OS	`Windows`	Info	Confirms Windows based hosting. Enables attacker to research Windows specific vulnerabilities and select appropriate exploitation tools and techniques.
HTTP TRACE Method	`Enabled (flagged as risky)`	Medium	The HTTP TRACE method echoes back the request headers to the client. This can be exploited via Cross Site Tracing (XST) to steal session cookies and authentication headers even when the HttpOnly flag is set on cookies.
HttpOnly Cookie Flag	`Not set on ASPSESSIONID cookie`	Medium	Session cookies without the HttpOnly flag are accessible to JavaScript running in the browser. A successful Cross Site Scripting (XSS) attack could directly read and exfiltrate the session token to steal authenticated sessions.
Filtered Ports	`999 ports filtered (no response)`	Info	999 out of 1000 scanned ports are filtered, indicating a firewall or security group is active. Only port 80 is intentionally exposed. This is consistent with a properly scoped AWS EC2 security group.
Application Title	`acuforum forums`	Info	The HTTP title confirms this is the Acunetix intentionally vulnerable forum application, consistent with the expected test environment.

Critical Observation: The most operationally significant finding from the Nmap scan is the combination of HTTP only transmission (no HTTPS on any port) with the absence of the HttpOnly flag on the ASP session cookie. This creates two compounding risks: first, the session cookie is transmitted in cleartext with every request and can be stolen via passive network sniffing; second, if an attacker can inject JavaScript via XSS,

the cookie can also be stolen from the browser directly. The HTTP TRACE method adds a third attack vector enabling Cross Site Tracing. Together these three findings create a layered credential and session theft risk that will be conclusively demonstrated in Task 3.

6. TASK 2 DIRECTORY ENUMERATION USING GOBUSTER AND FFUF

6.1 Objective and Background

The objective of Task 2 was to discover hidden, unlisted, or unlinked directories and files on the target web server through automated brute force enumeration. In a standard web application, not all directories and files are linked from the main navigation menu. Developers frequently leave administrative panels, backup files, configuration scripts, old versions of pages, and development artefacts in directories that are never referenced from the public facing pages. These resources are assumed to be private simply because they are not linked, but they remain fully accessible to anyone who knows or guesses the correct URL path.

Directory brute force enumeration works by systematically requesting a large list of candidate directory and file names from the web server and recording which ones return valid HTTP responses (such as 200 OK or 301 Moved Permanently) versus error responses such as 404 Not Found. A 301 redirect confirms the directory exists even if its contents require further investigation. A 403 Forbidden response is particularly valuable as it confirms the directory exists but access is restricted, making it a target for further exploitation attempts.

6.2 Tools Used: Gobuster and FFUF

Two separate tools were employed for this task to cross validate the results and demonstrate proficiency with multiple enumeration approaches. Both tools used the same wordlist (dirb/common.txt containing 4,613 entries) and produced consistent results, which provides strong confidence in the accuracy of the discovered directories.

Tool	Version	Description	Configuration Used
Gobuster	v3.8.2	Go based fast directory and file brute force tool by OJ Reeves. Multi threaded with configurable thread count.	50 threads, GET method, 404 as negative status code, 10 second timeout
FFUF	v2.1.0-dev	Fuzz Faster U Fool. Highly flexible web fuzzer with FUZZ keyword placeholder substitution in URL, headers, and body.	40 threads (default), FUZZ keyword in URL path, default status code matching
Wordlist	dirb/common.txt	Standard DIRB wordlist with 4613 common directory and file names used in web applications and web servers.	Full wordlist used, located at /usr/share/wordlists/dirb/common.txt

6.3 Gobuster: Command, Execution, and Output

The following command was executed in the Kali Linux terminal to perform the directory enumeration using Gobuster:

```
gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt -t 50
```

The flag **dir** activates directory enumeration mode. The flag **-u** specifies the target base URL. The flag **-w** provides the path to the wordlist. The flag **-t 50** sets 50 concurrent threads to maximise scanning speed while remaining stable.

```
┌──(kali㉿DESKTOP-P3C3923)-[~/Desktop/NITJ]
└─$ gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt -t 50
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://testasp.vulnweb.com/
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
_vti_cnf             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/_vti_cnf/]
aspnet_client        (Status: 301) [Size: 164] [--> http://testasp.vulnweb.com/aspnet_client/]
avatars              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/avatars/]
cgi-bin              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/cgi-bin/]
cgi-bin/             (Status: 403) [Size: 1233]
html                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/html/]
HTML                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/HTML/]
Images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/Images/]
images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/images/]
jscripts             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/jscripts/]
robots.txt           (Status: 200) [Size: 13]
t                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/t/]
T                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/T/]
templates            (Status: 301) [Size: 160] [--> http://testasp.vulnweb.com/templates/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 2: Gobuster directory enumeration results against testasp.vulnweb.com

```
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) and Christian Mehlmauer (@firefart)
===============================================================
[+] Url: http://testasp.vulnweb.com/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
_vti_cnf (Status: 301) [Size: 159] --> /testasp.vulnweb.com/_vti_cnf/
aspnet_client (Status: 301) [Size: 164] --> /testasp.vulnweb.com/aspnet_client/
avatars (Status: 301) [Size: 158] --> /testasp.vulnweb.com/avatars/
cgi-bin (Status: 301) [Size: 158] --> /testasp.vulnweb.com/cgi-bin/
cgi-bin/ (Status: 403) [Size: 1233]
html (Status: 301) [Size: 155] --> /testasp.vulnweb.com/html/
HTML (Status: 301) [Size: 155] --> /testasp.vulnweb.com/HTML/
Images (Status: 301) [Size: 157] --> /testasp.vulnweb.com/Images/
images (Status: 301) [Size: 157] --> /testasp.vulnweb.com/images/
jscripts (Status: 301) [Size: 159] --> /testasp.vulnweb.com/jscripts/
robots.txt (Status: 200) [Size: 13]
t (Status: 301) [Size: 152] --> /testasp.vulnweb.com/t/
T (Status: 301) [Size: 152] --> /testasp.vulnweb.com/T/
templates (Status: 301) [Size: 160] --> /testasp.vulnweb.com/templates/
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
```

6.4 FFUF: Validation and Cross Verification

To independently validate the Gobuster results, the same enumeration was performed using FFUF. The FUZZ keyword in the URL path is replaced with each wordlist entry during execution. FFUF produces colour coded output with response status, size, word count, and line count for each discovered path, providing richer data for analysis.

```
ffuf -u http://testasp.vulnweb.com/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

```
┌──(kali㉿DESKTOP-P3C3923)-[~/Desktop/NITJ]
└─$ ffuf -u http://testasp.vulnweb.com/FUZZ -w /usr/share/wordlists/dirb/common.txt

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
------------------------------------------------

 :: Method           : GET
 :: URL              : http://testasp.vulnweb.com/FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/dirb/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
------------------------------------------------

                        [Status: 200, Size: 3538, Words: 328, Lines: 46, Duration: 331ms]
_vti_cnf                [Status: 301, Size: 159, Words: 9, Lines: 2, Duration: 311ms]
aspnet_client           [Status: 301, Size: 164, Words: 9, Lines: 2, Duration: 311ms]
avatars                 [Status: 301, Size: 158, Words: 9, Lines: 2, Duration: 303ms]
cgi-bin                 [Status: 301, Size: 158, Words: 9, Lines: 2, Duration: 303ms]
cgi-bin/                [Status: 403, Size: 1233, Words: 73, Lines: 30, Duration: 321ms]
html                    [Status: 301, Size: 155, Words: 9, Lines: 2, Duration: 307ms]
HTML                    [Status: 301, Size: 155, Words: 9, Lines: 2, Duration: 314ms]
images                  [Status: 301, Size: 157, Words: 9, Lines: 2, Duration: 341ms]
Images                  [Status: 301, Size: 157, Words: 9, Lines: 2, Duration: 319ms]
jscripts                [Status: 301, Size: 159, Words: 9, Lines: 2, Duration: 325ms]
robots.txt              [Status: 200, Size: 13, Words: 2, Lines: 1, Duration: 330ms]
t                       [Status: 301, Size: 152, Words: 9, Lines: 2, Duration: 322ms]
T                       [Status: 301, Size: 152, Words: 9, Lines: 2, Duration: 322ms]
templates               [Status: 301, Size: 160, Words: 9, Lines: 2, Duration: 317ms]
:: Progress: [4614/4614] :: Job [1/1] :: 124 req/sec :: Duration: [0:00:37] :: Errors: 0 ::
```

Figure 3: FFUF web fuzzer results confirming all directories found by Gobuster

6.5 Discovered Directories: Full Analysis

Both tools discovered an identical set of 14 accessible paths. Each discovered path is analysed below with its HTTP response code, likely purpose, and associated security risk:

Directory Path	HTTP Status	Likely Purpose	Risk Level	Security Concern
`/_vti_cnf/`	301 Redirect	Microsoft FrontPage server extensions configuration folder	Medium	May expose server configuration data and internal path information used by FrontPage publishing tools
`/aspnet_client/`	301 Redirect	ASP.NET framework client side scripts	Low	Confirms the application technology stack. May contain outdated JavaScript libraries with known vulnerabilities

`/avatars/`	301 Redirect	User uploaded avatar and profile image storage	Medium	If directory listing is enabled, all user avatar images are enumerable. File upload endpoint may be a path traversal vector
`/cgi-bin/`	301 then 403	Common Gateway Interface script directory	High	The 301 redirect followed by 403 Forbidden confirms the directory exists but access is restricted. CGI scripts are a historically common source of remote code execution vulnerabilities
`/html/`	301 Redirect	Static HTML content folder	Low	May contain static documentation, legacy pages, or developer notes that were not intended to be publicly accessible
`/HTML/`	301 Redirect	Case variant of /html/ on case insensitive server	Info	Confirms the server is running on Windows with a case insensitive file system, consistent with IIS behaviour on NTFS
`/Images/` and `/images/`	301 Redirect	Application image asset storage	Low	Case insensitive file system confirms Windows IIS server. Both paths resolve to same directory. Directory listing status unknown
`/jscripts/`	301 Redirect	JavaScript libraries and client side scripts	Medium	Source review of JavaScript files may reveal API endpoints, business logic, hardcoded credentials, or developer comments disclosing internal architecture details
`/robots.txt`	200 OK	Search engine crawler exclusion file	Low	A publicly readable robots.txt effectively acts as a sitemap for attackers by explicitly listing paths the owner wishes to hide from search engines. Disallow entries reveal sensitive locations
`/templates/`	301 Redirect	Application page templates and layout files	Medium	Template files may reveal the internal structure of the application and could be a target for server side template injection attacks
`/t/ and /T/`	301 Redirect	Unknown short path, possibly a shortcut or alias	Medium	Ambiguous directories warrant further investigation. Short generic names sometimes indicate auto generated or temporary folders that may contain sensitive data

Key Observation: The discovery of 14 accessible paths from a wordlist of only 4,613 entries represents a substantial attack surface. The most concerning finding is the /cgi-bin/ directory which returns a 403 Forbidden response confirming its existence despite access restrictions. Historically, CGI scripts written in

Perl or Shell have been among the most frequently exploited components in web servers. The /_vti_cnf/ directory is a remnant of Microsoft FrontPage server extensions and its presence indicates the server may have legacy configuration that has not been reviewed or hardened. The case insensitive responses to both /Images/ and /images/ conclusively confirm a Windows NTFS file system, which is consistent with the IIS 8.5 identification from Task 1. The /jscripts/ directory is particularly interesting for further manual investigation as JavaScript source files frequently contain hardcoded API keys, internal endpoint URLs, and developer comments.

7. TASK 3 PLAINTEXT CREDENTIAL CAPTURE USING WIRESHARK

7.1 Objective and Background

The objective of Task 3 was to intercept live network traffic during an active login session on the target application and extract the credentials transmitted over the network. This task directly demonstrates the real world impact of one of the most serious and commonly overlooked web application security failures: the use of unencrypted HTTP for any page that handles authentication.

When a web application uses HTTP instead of HTTPS, all data travelling between the user's browser and the web server passes through the network in plaintext. This includes form submissions containing usernames and passwords, session cookies that grant authenticated access, personal data entered into forms, and all page content delivered by the server. Any device or software positioned on the network path between the user and the server can read this data without any decryption. This type of attack is commonly called a passive network sniffing or eavesdropping attack.

The attack does not require the attacker to actively interfere with the connection. It requires only that the attacker can observe the network traffic, which is trivially achievable in shared network environments such as coffee shop WiFi, university campus networks, and corporate environments with access to a network switch or router. In cloud hosted environments, an attacker who has compromised any other virtual machine on the same physical host may be able to observe traffic depending on the network virtualisation configuration.

7.2 Tool Overview: Wireshark

Wireshark is the world's most widely used open source network protocol analyser, maintained by the Wireshark Foundation and a global community of contributors. It captures all packets traversing the selected network interface in real time and provides a rich graphical interface for deep inspection of individual packets across more than three thousand network protocols. Wireshark supports powerful display filters written in its own filter expression language that allow the analyst to isolate specific types of traffic from millions of captured packets.

For this task, Wireshark was configured to capture all traffic on the eth0 network interface (the primary internet facing adapter in the Kali Linux environment). A display filter of **http.request.method == "POST"** was applied after capture to isolate only HTTP POST requests, as login and registration form submissions in web applications are universally transmitted via HTTP POST. This filtered the many thousands of captured packets down to only the small number of packets relevant to the credential submission.

7.3 Step by Step Methodology

Step 1: Prepare the Capture Environment

Wireshark was launched with superuser privileges using the command **sudo wireshark** from the Kali Linux terminal. Elevated privileges are required to access the network interface in promiscuous mode for raw packet capture. The eth0 interface was selected as the capture interface and packet capture was started before any browser activity to ensure no traffic was missed.

Step 2: Launch Firefox Browser

Firefox ESR was launched from a separate terminal window using the **firefox** command. The browser was navigated to the target application at http://testasp.vulnweb.com/ to begin the user interaction that would generate the credential bearing POST requests.

Step 3: Register a New Account

A new user account was registered on the target application by navigating to the registration page at http://testasp.vulnweb.com/register.asp and completing the registration form. The registration form collected a username, real name, email address, and password, all of which were transmitted to the server via an HTTP POST request that was captured by Wireshark.

Step 4: Perform Login

After registration, the login page at http://testasp.vulnweb.com/Login.asp was accessed and the credentials were submitted. This generated a second HTTP POST request containing the username and password fields, which was also captured by Wireshark.

Step 5: Apply Wireshark Display Filter

After completing the browser interactions, the Wireshark display filter **http.request.method == "POST"** was applied in the Wireshark filter bar. This immediately reduced the display to only the packets matching HTTP POST requests, making the credential bearing packets immediately visible and easy to inspect.

Step 6: Inspect Packet Contents and Extract Credentials

Each captured POST packet was selected and its payload was inspected using Wireshark's packet detail pane. The form data submitted with each request was visible in full plaintext in the packet payload, including all username and password fields.

7.4 Proof of Concept: Wireshark Capture Active

The screenshot below shows Wireshark running with an active packet capture on the eth0 interface alongside the Firefox browser opened to the target application. This confirms both tools were running simultaneously, with Wireshark recording all network traffic generated by the browser session.

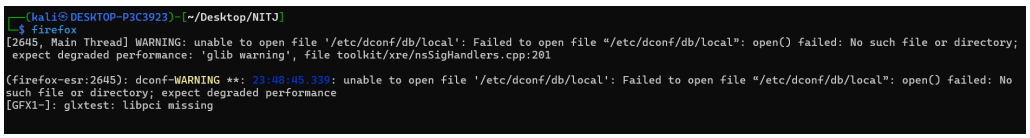

Figure 4: Wireshark packet capture active on eth0 interface alongside Firefox browser

7.5 Target Application Registration Page

The screenshot below shows the acuforum registration page. A new account was successfully created using the test credentials. At this point, Wireshark was actively capturing all network traffic including the form submission that carried these credentials in plaintext across the network.

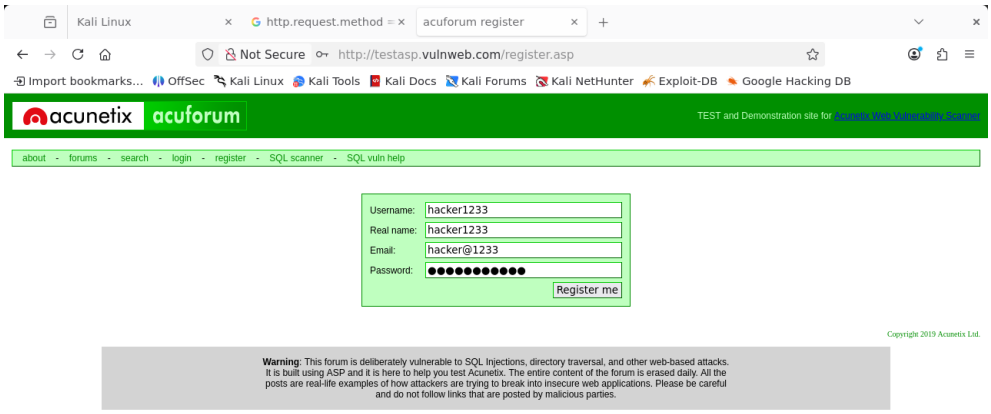

Figure 5: Target application registration page at testasp.vulnweb.com showing completed registration form

7.6 Wireshark POST Filter Applied

The Wireshark display filter **http.request.method == "POST"** was applied after completing the browser interactions. This filter expression matches only those HTTP packets where the request method field equals POST, immediately isolating the registration and login submission packets from all other network traffic. The screenshot below shows the filtered Wireshark view with the relevant credential bearing packets visible.

```
http.request.method == "POST"
```

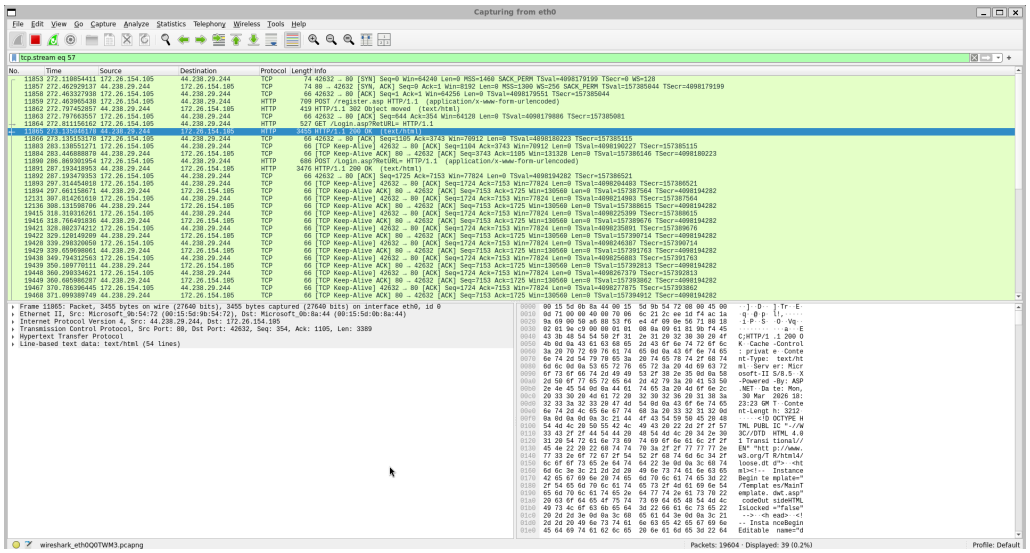

Figure 6: Wireshark display with POST filter applied, isolating credential submission packets

7.7 Full Captured Packet Data

The complete HTTP POST packet data captured by Wireshark for both the registration and login requests is reproduced below in full. The credential fields are clearly visible in the packet body with no encryption whatsoever:

Registration Request (POST to /register.asp):

```
POST /register.asp HTTP/1.1
Host: testasp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Content-Type: application/x-www-form-urlencoded
Content-Length: 81
Origin: http://testasp.vulnweb.com
Referer: http://testasp.vulnweb.com/register.asp
Cookie: ASPSESSIONIDCQDABSCC=GNPAOENAJCMDFIDFDCIDLFKO

tfUName=hacker1233&tfRName;=hacker1233&tfEmail;=hacker%401233&tfUPass;=hacker%401233
^^^^^^^^^^^^^^^^^^^
*** FULL CREDENTIALS CAPTURED IN PLAINTEXT ***
```

Login Request (POST to /Login.asp):

```
POST /Login.asp?RetURL= HTTP/1.1
Host: testasp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 48
Origin: http://testasp.vulnweb.com
Referer: http://testasp.vulnweb.com/Login.asp?RetURL=
Cookie: ASPSESSIONIDCQDABSCC=GNPAOENAJCMDFIDFDCIDLFKO

tfUName=hacker123&tfUPass;=hacker1233%40gmail.com
^^^^^^^^^^^^^^^^^^^^^^^^^
*** USERNAME AND PASSWORD BOTH VISIBLE AS CLEARTEXT ***
```

```
POST /register.asp HTTP/1.1
Host: testasp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 81
Origin: http://testasp.vulnweb.com
Connection: keep-alive
Referer: http://testasp.vulnweb.com/register.asp
Cookie: ASPSESSIONIDCQDABSCC=GNPAOENAJCMDFIDFDCIDLFKO
Upgrade-Insecure-Requests: 1
Priority: u=0, i

tfUName=hacker1233&tfRName=hacker1233&tfEmail=hacker%401233&tfUPass=hacker%401233
HTTP/1.1 302 Object moved
Cache-Control: private
Content-Type: text/html
Location: Login.asp?RetURL=
Server: Microsoft-IIS/8.5
X-Powered-By: ASP.NET
Date: Mon, 30 Mar 2026 18:23:22 GMT
Content-Length: 138

<head><title>Object moved</title></head>
<body><h1>Object Moved</h1>This object may be found <a HREF="Login.asp?RetURL=">here</a>.</body>
```

Figure 7: Wireshark packet detail view showing full plaintext credential data extracted from captured HTTP POST packets

7.8 Decoded Credential Analysis

The credentials in the POST body are URL encoded, which is a standard HTTP data formatting convention. URL encoding is categorically not encryption. It is a trivial reversible encoding where characters such as the at symbol are replaced with their percent encoded equivalents (the at symbol becomes %40). Any tool or programmer can reverse this encoding in milliseconds. The table below shows the complete decoded

credentials extracted from both captured POST requests:

Request Type	Field Name	URL Encoded Value	Decoded Plaintext Value	Field Purpose
Registration	`tfUName`	`hacker1233`	`hacker1233`	Desired username for new account
Registration	`tfRName`	`hacker1233`	`hacker1233`	Real or display name for profile
Registration	`tfEmail`	`hacker%401233`	`hacker@1233`	Email address for account
Registration	`tfUPass`	`hacker%401233`	`hacker@1233`	Password chosen for new account
Login	`tfUName`	`hacker123`	`hacker123`	Username submitted at login page
Login	`tfUPass`	`hacker1233%40gmail.com`	`hacker1233@gmail.com`	Password submitted at login page

Critical Security Observation: The Wireshark capture conclusively demonstrates that the complete set of user registration and login credentials is transmitted across the network in cleartext with zero protection. The URL encoding of the at symbol (%40) provides absolutely no security benefit and any attacker observing the network can decode it instantly. Additionally, the session cookie ASPSESSIONIDCQDABSCC=GNPAOENAJCMDFIDFDCIDLFKO is visible in every captured request. An attacker who intercepts this cookie can perform a session hijacking attack, impersonating the authenticated user without even needing the username and password. This cookie is transmitted in cleartext in every single HTTP request made by the browser to the server for the duration of the session, providing a continuous opportunity for theft.

8. CONSOLIDATED VULNERABILITY ASSESSMENT

The following table consolidates all security vulnerabilities identified across all three beginner level tasks. Severity ratings are assigned using the CVSS v3 framework principles adapted for the context of the assessment:

ID	Vulnerability Name	Severity	CVSS Approx.	Discovered In	Evidence
V01	Insecure HTTP Transmission (No HTTPS)	Critical	9.1	Task 1 and 3	Port 80 open. No HTTPS port detected. Credentials in cleartext.
V02	Plaintext Credential Submission via HTTP POST	Critical	9.4	Task 3	Full username and password captured in Wireshark POST packet body.
V03	Session Cookie Transmitted in Cleartext	High	8.1	Task 3	ASPSESSIONID cookie visible in all captured HTTP request packets.
V04	End of Life Web Server Software (IIS 8.5)	High	7.5	Task 1	Nmap detected Microsoft IIS 8.5, EOL since October 2023.
V05	Missing HttpOnly Flag on Session Cookie	Medium	6.1	Task 1	Nmap NSE script reported httponly flag not set on ASPSESSIONID.
V06	HTTP TRACE Method Enabled	Medium	5.4	Task 1	Nmap http-methods script flagged TRACE as a potentially risky method.
V07	Excessive Directory Exposure via Enumeration	Medium	5.3	Task 2	14 directories enumerated. Sensitive folders including CGI and FrontPage exposed.
V08	CGI Directory Present and Confirmed	High	7.3	Task 2	cgi-bin confirmed via 301 redirect. 403 on direct access confirms existence.
V09	FrontPage Server Extensions Exposed	Medium	4.8	Task 2	_vti_cnf directory discovered, indicating FrontPage extensions are active.

9. RISK ANALYSIS AND IMPACT ASSESSMENT

Understanding the vulnerabilities in isolation is insufficient for a complete security assessment. The true impact must be considered from the perspective of what an attacker could achieve by chaining these vulnerabilities together in a real world attack scenario against a live application with real users.

Attack Scenario 1: Passive Credential Harvesting

An attacker positioned on the same network as a legitimate user (such as a shared WiFi access point) can run Wireshark passively without sending any packets to the target server. The attacker simply records all traffic and filters for HTTP POST requests to testasp.vulnweb.com. Every user who logs in or registers will have their credentials immediately captured. This attack is completely silent, requires no special skills beyond knowing how to use Wireshark, and leaves no logs on the server because no unusual requests are being made. The credentials can then be used to log into the forum as those users.

Attack Scenario 2: Session Hijacking Without Credentials

Even if a user changes their password, an attacker who has already captured their session cookie (the ASPSESSIONID value visible in Task 3) can inject that cookie into their own browser and access the application as the victim for the duration of the active session. This does not require the password and cannot be prevented by a password change unless the server also invalidates all existing sessions.

Attack Scenario 3: Chained Discovery to Exploitation

The directories discovered in Task 2 provide a roadmap for deeper exploitation. An attacker would follow Task 2 findings with manual investigation of the /jscripts/ directory to review JavaScript source code, attempt to access configuration files within the /_vti_cnf/ directory, and probe the /cgi-bin/ endpoint for remotely executable scripts. Combined with the session hijacking capability demonstrated in Task 3, an attacker who gains admin level access through any of these vectors could achieve complete application compromise.

10. DETAILED RECOMMENDATIONS AND REMEDIATION PLAN

The following remediation recommendations are listed in order of priority. Each recommendation directly addresses one or more identified vulnerabilities and is accompanied by specific implementation guidance:

R1: Implement HTTPS with TLS 1.2 or Higher	Critical	Addresses: V01, V02, V03

Obtain a valid TLS certificate from a trusted Certificate Authority such as Let's Encrypt (free) or a commercial CA. Configure IIS to bind HTTPS on port 443 using the certificate. Configure a permanent 301 redirect from all HTTP traffic to HTTPS so that no user can accidentally connect over the unencrypted channel. Add the HTTP Strict Transport Security (HSTS) response header with a minimum max-age value of 31536000 seconds (one year) to instruct browsers to always use HTTPS for this domain. Disable TLS 1.0 and TLS 1.1, permitting only TLS 1.2 and TLS 1.3 to prevent downgrade attacks.

R2: Upgrade Web Server to Supported Software	High	Addresses: V04

Migrate the application from Microsoft IIS 8.5 to IIS 10.0 on Windows Server 2022, which receives active security support from Microsoft until 2031. Alternatively, consider migrating to a modern web application hosting platform. After upgrading, apply all available Windows Server and IIS security patches immediately and establish a monthly patch management schedule to ensure ongoing compliance with the latest security updates.

R3: Set HttpOnly and Secure Flags on All Session Cookies	High	Addresses: V05

Configure IIS to set the HttpOnly attribute on all session cookies by adding the following entry to the web.config file under the system.web/httpCookies element: httpOnlyCookies='true'. Also set the requireSSL attribute to 'true' to add the Secure flag, preventing cookies from being transmitted over HTTP connections. Additionally consider setting the SameSite cookie attribute to Strict or Lax to mitigate Cross Site Request Forgery risks.

R4: Disable the HTTP TRACE Method	Medium	Addresses: V06

Disable the HTTP TRACE method on the IIS server by adding a verb restriction in the web.config handlers or through the IIS Manager request filtering settings. Remove TRACE from the list of allowed HTTP verbs. This prevents Cross Site Tracing attacks which could bypass the HttpOnly cookie protection.

R5: Implement Access Controls on Sensitive Directories

High | Addresses: V07, V08, V09

Review all discovered directories and apply appropriate access controls. Remove the FrontPage server extensions (_vti_cnf) if they are not actively required as they represent legacy functionality with a poor security track record. Restrict the cgi-bin directory with IP allowlisting if CGI scripts are needed, or remove the directory entirely if no CGI functionality is in use. Disable directory browsing globally in IIS to prevent attackers from listing directory contents. Set explicit deny rules for sensitive system directories.

R6: Implement Multi Factor Authentication

Medium | Addresses: V01, V02

Add a second authentication factor to the login process so that a stolen password alone is insufficient to access an account. This can be implemented using Time based One Time Passwords (TOTP) via apps such as Google Authenticator or Authy, SMS based one time codes sent to the registered phone number, or email based verification codes. Even if an attacker captures credentials via network sniffing, they will be unable to complete authentication without the second factor.

R7: Implement Login Rate Limiting and Account Lockout

Medium | Addresses: V01, V02

Implement a mechanism to limit the number of failed login attempts from a single IP address or for a single account within a defined time window. After five consecutive failed attempts, temporarily lock the account for fifteen minutes and send an alert notification to the account email address. After ten consecutive failures, require manual administrator intervention to unlock. Log all failed login attempts with timestamp and source IP address for security monitoring and incident response purposes.

R8: Review and Sanitise robots.txt

Low | Addresses: V07

Remove all Disallow entries from robots.txt that reference directories containing sensitive functionality. Security through obscurity is not a valid security control. Robots.txt is publicly readable and its Disallow entries serve as a directory listing for attackers. Sensitive paths should be protected by proper access controls rather than exclusion from search engine crawlers.

R9: Conduct Regular Security Assessments

Medium | Addresses: All

Establish a programme of regular security assessments to identify new vulnerabilities as the application evolves. At minimum, conduct a penetration test annually or following any significant changes to the application or infrastructure. Use automated vulnerability scanning tools such as Nessus or OpenVAS on a monthly schedule to catch newly disclosed CVEs in the software stack. Maintain a vulnerability disclosure programme to allow security researchers to responsibly report issues.

11. TOOLS AND TECHNOLOGIES REFERENCE

The following table provides a comprehensive reference of all tools used during this assessment, including version information, official project URLs, and the specific purpose each tool served in the assessment:

Tool Name	Version	Category	Use in This Assessment	Official Source
Nmap	7.98	Port Scanner	TCP port enumeration, service version detection, OS fingerprinting, NSE default script execution for banner grabbing and header analysis	`https://nmap.org/`
Gobuster	3.8.2	Directory Fuzzer	Web directory brute force enumeration using the dirb common.txt wordlist with 50 concurrent threads	`github.com/OJ/gobuster`
FFUF	2.1.0	Web Fuzzer	Cross validation of directory enumeration using FUZZ keyword URL substitution to confirm all directories found by Gobuster	`github.com/ffuf/ffuf`
Wireshark	Latest	Network Analyser	Live packet capture on eth0 interface, HTTP POST filter application, packet payload inspection and credential extraction	`https://www.wireshark.org/`
Firefox ESR	140.0	Web Browser	Generating authentic HTTP traffic to the target application including registration and login form submissions for capture by Wireshark	`mozilla.org/firefox/`
Kali Linux	2024	Operating System	Primary testing environment providing all security tools pre installed and the network interface for packet capture	`https://www.kali.org/`

12. LEARNING OUTCOMES AND CONCLUSION

12.1 Technical Skills Acquired

The completion of the three beginner level tasks provided hands on practical experience with a range of industry standard penetration testing tools and techniques. Each task built progressively on the skills developed in the previous one, creating a coherent reconnaissance to exploitation workflow that mirrors real world penetration testing practice.

Nmap Port Scanning

Gained practical proficiency in executing Nmap scans with version detection and script scanning flags. Learned to interpret Nmap output including service banners, OS detection results, NSE script output, and HTTP cookie analysis results. Understood the significance of each open port in the context of the overall attack surface.

Web Directory Enumeration

Developed skill in using both Gobuster and FFUF for web content discovery. Understood the difference in approach between the two tools and the value of cross validating results using multiple independent tools. Learned to interpret HTTP response codes including the significance of 200, 301, 403, and 404 responses in the context of directory enumeration.

Network Traffic Analysis with Wireshark

Gained practical experience configuring Wireshark for live packet capture and applying targeted display filters to isolate specific traffic types. Developed the ability to navigate Wireshark's packet detail pane to extract HTTP payload data and interpret URL encoded form submissions. Understood the critical security distinction between URL encoding and cryptographic encryption.

Security Report Writing

Developed the ability to document technical security findings in a professional format appropriate for submission to a security team or client. Learned to structure findings with clear descriptions, evidence, impact assessments, and actionable remediation guidance.

Penetration Testing Methodology

Understood and applied the four phase penetration testing lifecycle of planning, discovery, exploitation, and reporting. Appreciated the importance of following a structured methodology to ensure comprehensive coverage and reproducible results.

12.2 Key Security Principles Reinforced

- Transport layer encryption (HTTPS) is not optional for any application that handles user authentication or personal data. The moment credentials travel over HTTP, the application has failed its fundamental duty to protect its users.
- End of life software on internet facing servers is an unacceptable risk because vulnerabilities discovered after the end of life date will never be patched by the vendor.
- Security through obscurity is not a security control. Directories that are not linked from the main application are trivially discoverable through automated enumeration in a matter of minutes using freely available tools.
- Defence in depth requires multiple overlapping controls. A single missing control such as HTTPS creates a single point of failure that undermines every other security measure the application may have implemented.
- Proof of concept documentation is essential in security work. Demonstrating the real world exploitability of a vulnerability with captured evidence is far more compelling than a theoretical description and provides unambiguous justification for remediation investment.

12.3 Final Summary Statistics

Tasks Completed	Vulnerabilities Found	Directories Found	Critical Findings	Credentials Captured

3 of 3	**9**	**14**	**2**	**2 Sets**

12.4 Conclusion

This beginner level security assessment of http://testasp.vulnweb.com/ successfully demonstrated the practical application of three core penetration testing disciplines in a controlled and ethically appropriate environment. All tasks were completed to the full requirements specified in the ShadowFox Cyber Security Internship Task List.

The findings of this assessment powerfully illustrate that even a basic, beginner level penetration test conducted with freely available open source tools can uncover critical security vulnerabilities in a matter of hours. The complete capture of user credentials in Task 3 was achieved with a single command and no advanced exploitation technique, requiring only that the application used HTTP instead of HTTPS. This underscores the fundamental importance of getting security basics right before addressing more sophisticated threat vectors.

This internship experience has provided an invaluable foundation in offensive security thinking and the practical use of professional grade security tooling. The structured programme at ShadowFox provided the right combination of guided task objectives and hands on technical execution to build genuine, demonstrable skills in the field of cyber security.

Arun Kumar
Cyber Security Intern, ShadowFox
arungaming1973@gmail.com
Date: March 31, 2026